Activity 2.2: Presence-Based Attendance Logic

1. Activity Overview & Objective

This activity focuses on translating face tracking data into meaningful attendance records. Instead of marking attendance based on a single frame detection, the system uses **presence-based logic**, where attendance is confirmed only when a participant remains visible for a minimum duration during the session.

> **Fairness & False-Positive Mitigation:** Presence duration is calculated using timestamps collected during the tracking phase. Intermittent appearances are accumulated to ensure fairness and to prevent false attendance marking caused by brief or accidental detections. Once the cumulative presence exceeds a predefined threshold, the participant is marked as present.

2. Presence Accumulation & Verification Pipeline

Attendance results are stored in structured logs, including participant identifiers and cumulative presence duration. This approach improves accuracy, reduces false positives, and provides reliable attendance records suitable for classroom analytics and reporting:

Verification Stage	Mechanism & Rule Logic	Reliability & Fairness Benefit
Identity Matching	Periodic facial verification against `student_db` via `DeepFace.find()`.	Associates tracked bounding-box trajectories with verified student identifiers.
Timestamp Accumulation	Summation of active frame intervals (`Δt = t_current - t_last_seen`) across tracking segments.	Accumulates intermittent visibility so temporary camera occlusions do not reset a student's progress.
Minimum Duration Threshold	Rule check verifying `cumulative_duration >= min_presence_threshold`.	Filters out transient passersby, false-positive flashes, or brief walk-ins from being marked present.
Structured Attendance Logging	Persistence of verified `student_id`, `presence_duration`, and confirmation status.	Produces audit-ready session logs for downstream engagement analytics and institutional reporting.

3. Backend Implementation (`backend/core/face_recognition.py`)

To bind tracked presences to enrolled learners while keeping compute overhead manageable, `FaceRecognizer` utilizes `DeepFace` against a local reference database directory (`student_db`). Because deep facial embedding extraction is compute-intensive, recognition is invoked sparingly alongside continuous tracker timestamps to accumulate verified presence duration.

```
# File: backend/core/face_recognition.py
from deepface import DeepFace
import cv2
import numpy as np
import os

class FaceRecognizer:
    def __init__(self, db_path="student_db"):
        self.db_path = db_path
        if not os.path.exists(db_path):
            os.makedirs(db_path)

    def register_student(self, img_path, student_id):
        # In a real app, we'd store embeddings.
        # DeepFace.find uses a folder of images as DB.
        pass

    def recognize(self, frame):
        try:
            # Save frame temporarily or pass numpy array directly if supported
(DeepFace supports numpy)
            # DeepFace.find requires a db_path with images.
            # If no DB yet, we can't recognize.
            if not os.listdir(self.db_path):
                return []

            # This is computing intensive, so we should run it sparingly
            dfs = DeepFace.find(
                img_path=frame,
                db_path=self.db_path,
                silent=True,
                enforce_detection=False
            )
            return dfs
        except Exception as e:
            print(f"Recognition error: {e}")
            return []
```

4. Key Engineering & Architectural Highlights

- **Decoupled Tracking & Recognition Cadence:** Because `DeepFace.find()` is computationally expensive, running recognition sparingly while relying on lightweight frame-to-frame spatial tracking for timestamp accumulation preserves real-time frame rates.
- **Graceful Empty-Database Guardrails:** Checking `if not os.listdir(self.db_path): return []` and setting `enforce_detection=False` prevents runtime crashes when initializing new classroom cohorts or processing partially occluded frames.
- **Cumulative Fairness Guarantee:** Aggregating intermittent appearances into a single cumulative duration metric ensures students are evaluated fairly across the entire session lifecycle rather than penalized by momentary posture shifts or camera dropouts.